Distributed Systems Simplified

CAP Theorem:

Choosing between perfection and availability.

AUTHOR:

Niv Rave

Github: https://github.com/NivRave/Distributed-Systems-Simplified

The Choice You Can't Avoid

The "P" is Not a Choice

Most people think CAP means "Pick 2 out of 3." It’s actually simpler and harsher.

The Reality:
Network failures (Partitions) are inevitable. Cables get cut; routers crash.

The Constraint:
Since P is mandatory in distributed systems, your only real choice is what happens when the network breaks.

The Scaling Wall:

- **CP (Consistency)**: Stop the world to prevent wrong data.
- **AP (Availability)**: Keep serving, even if the data is stale.

The Fix: Align your architecture with your business needs.

The Architectural Solution

Strategy 1 vs. Strategy 2

- **AP (Availability + Partition Tolerance):** Prioritize uptime. If Node A can't talk to Node B, Node A accepts the write anyway.
- **CP (Consistency + Partition Tolerance):** Prioritize correctness. If Node A can't talk to Node B, it refuses the write to prevent a "Split Brain".

The Trade-off:

AP => Better User Experience, Stale Data.

CP => Perfect Data, Higher Error Rate.

Code: The Strategy Strategy

Modeling the Decision Logic

In our Go simulation, a Node isn't just a database; it's a decision engine. We use a Mode flag to dictate how the node handles a failed replication attempt.

- sync.RWMutex: Ensuring thread safety while we evaluate the network state.
- Mode Switching: The same code behaves differently based on whether it's configured for a Bank (CP) or a Feed (AP).

Below is the implementation under *node.go*:

```go
type Node struct {
    ID          string
    Mode        string // "CP" or "AP"
    Data        string
    Partitioned bool
    mu          sync.RWMutex
}
```

Code: The CP Logic (The Bank)

Safety First: Aborting the Write

In CP Mode, if we can't guarantee that every node will see the change, we refuse to make the change.

- **The Action**: If replication fails, we return a 503 Service Unavailable.
- **The Result**: No "Split Brain". Your bank balance stays consistent across the entire cluster.

Below is the implementation under *node.go*:

```go
// CP Strategy: Consistency above all
if n.Partitioned {
    // We cannot guarantee the other node is in sync.
    // To remain CONSISTENT, we must become UNAVAILABLE.
    return fmt.Errorf("503 Service Unavailable: Quorum Failed")
}
n.Data = newValue // Only happens if network is healthy
```

Code: The AP Logic (The Feed)

Keep Serving: Embracing the Lag

In AP Mode, we acknowledge the network is broken but decide that a response, even a stale one - is better than an error.

- **The Action**: Log a [WARNING], but persist the data locally.
- **The Result**: The user can still post their comment, even if their friends can't see it yet.

Below is the implementation under *node.go*:

```
// AP Strategy: Availability above all
if n.Partitioned {
    // Network is down, but we don't want to show an error.
    // We stay AVAILABLE, even if we become INCONSISTENT.
    log.Println("[WARNING] Partition active. Writing locally.")
}
n.Data = newValue // Always proceed
```

Code: Visualizing the Execution

The Simulation in Action - Try it out!

We orchestrate two distinct scenarios to see the divergence:

- **Scenario A (Social Media)**: We cut the cable, write a "Status Update," and show that Node A and Node B now disagree.
- **Scenario B (The Bank)**: We cut the cable, try a "Transfer," and show the system successfully blocking the operation.
- **Graceful Shutdown**: Using our standard context.Context pattern to ensure the simulation cleans up properly.

Below is the implementation under *simulation.go*:

```go
func (s *Simulation) Run(ctx context.Context) {
    fmt.Println("Scenario 1: AP Mode (Social Media)")
    s.NodeA.Mode = "AP"
    s.Network.Disconnect() // Simulate "P"
    s.NodeA.Write("Status: Busy") // Returns Success

    fmt.Println("Scenario 2: CP Mode (Banking)")
    s.NodeA.Mode = "CP"
    s.NodeA.Write("Transfer: $100") // Returns Error!
}
```

Code: Stress Testing the System

Analyzing the Logs

Below we can see the detailed output of the program, using the simulation for both the AP and CP scenarios:

```
=== CAP Theorem Simulation ===

--------------------------------------------------
Scenario 1: Availability (AP) - The Social Network
Goal: Keep the service UP, even if data is stale.
--------------------------------------------------

[Step 1] Normal Operation (Network Healthy)
[Node-A] Received Write Request ('Status: Online')...
   -> [Success] [AP Decision] Replication OK.
   Node-B sees: 'Status: Online' (Consistent)

[Step 2] Network Partition Occurs (Cable Cut)

[Network] [Partition] Connection severed between Node-A and Node-B...

[Step 3] Client updates status on Node-A
[Node-A] Received Write Request ('Status: Busy')...
   -> [Warning] [AP Decision] Replication Failed, but proceeding (Availability First!).

[Step 4] Reading from both nodes
   Node-A (Local):  'Status: Busy'
   Node-B (Remote): 'Status: Online'

Result: System is AVAILABLE (Write succeeded), but INCONSISTENT.

--------------------------------------------------
Scenario 2: Consistency (CP) - The Bank
Goal: Prevent conflicting data, even if we must go down.
--------------------------------------------------

[Step 1] Network Partition Occurs

[Network] [Partition] Connection severed between Node-A and Node-B...

[Step 2] Client attempts transaction on Node-A
[Node-A] Received Write Request ('Balance: $500')...
   -> [Error] [CP Decision] Replication Failed. Aborting Write to serve Consistency.
   Client received error: 503 Service Unavailable: Replication Quorum Failed

[Step 3] Verifying Data Integrity
   Node-A State: 'InitialState'
   Node-B State: 'InitialState'

Result: System is CONSISTENT (Data matches), but UNAVAILABLE for writes.
```

Summary & Takeaways

P IS NOT OPTIONAL

You cannot choose to have a perfect network. You can only choose your response to its failure.

AP FOR UX

If a "service down" page is worse than slightly stale data (e.g., Social Media), choose AP.

CP FOR FINANCES

If data accuracy is more important than uptime (e.g., Banking), choose CP.

THE PACELC EXTENSION

CAP only describes behavior during a partition. PACELC covers the trade-offs during normal operation (Latency vs. Consistency).

Want to dive deeper?

Explore the full source code and deep-dive documentation on my GitHub repository or connect via LinkedIn.

GitHub LinkedIn

LIKE • COMMENT • SHARE